Specification of the Sonnet Programming Language

Version 0.1.0 dated 10.06.2026

StefanValentinT

Abstract

This specification details the syntactic form and evaluation of programs written in the Sonnet programming language. It details, how programs are represented, how these representations map to an understnading of terms, and how these terms get evaluated. In doing so it defines the requirements for a conforming implementation, thus establishing a single source of truth for the meaning of any program topromote the portability of the language itself.

Contents

1. Introduction

Sonnet ought to be beautiful in its simplicity, pristine in its architecture, bright-shining as the ivory tower, with a long spiral staircase winding its way all the way to the top, twisting round and round the tower. Sonnet shall be there for everyone, shall be easy to use because of its simplictly, but limitless in its reach. Sonnet shall bring back the simple, arcane joy of programming.

Therefore, the specification is written in a way to facilitate both unambiguity and accessibility for everyone. Whenever these goals conflicted, we opted to add important information using a Note, or, where needed, to provide additional reiteration or exemplification marked with an Example, whilst not compromising the precise actual specification. Both shall be regarded as non-authoritative for the meaning of the language and may only aid in its understanding.

1.1. Scope

We first detail the actual representation of Sonnet programs, before specifying their syntactical structure. Thereafter the constraints and requirements enacted upon the maening of a program are detailed and its evaluation via the reduction of terms. The means by which a program may interface with the

surrounding world are described and lastly a collection of definitions is given that a conforming implementation shall provide as minimal, opaque "atoms" of computation.

1.2. Limitations

No bound is given for the complexity of a program that may exceed the capabilities of a particular evaluator, compiler or processor. Although such limits seem to be necessary in practice, especially for data, we also do not demand minimal abilities of a system to not constrain the language by artificial limits.

Optimization. The means by which a conforming evaluator is invoked or the way it evaluates a given program are left unspecified to allow for greater freedom of optimization. Only directly observable behaviour of the language itself shall be specified.

2. Conformance & Specification

The following definitions of verbs shall only apply within the context of a constraint on the implementation.

The use of "shall" denotes a mandatory requirement on an implementation, while "shall not" and "may not" declare a strict prohibition.

"May" is used to specify that some behaviour is conformant to the specification, noticeably it is not exclusive, signifying multiple interpretations may be within the bounds of conformance. "Need not" is used to explicitly denote the non-requiredness of a particular property.

A "conforming" implementation is any one that abides to all requirements laid out by this specification, including those requiring the rejection of invalid or erroneous programs. Similarily, a conforming program is one not rejected by a conforming implementation.

A implementation shall be accompanied by a clear definition of the behaviour in all cases of implementation-defined behaviour in this specification, for all cases where the characteristics do not match the recommendations of this specifications non-authorative text.

3. Definitions

The following definitions apply to the specification. Other terms are defined when they first occur or in a more appropriate section; when a term is defined it is being written in *italic* type.

- Observable **behaviour** is all action that influences the world. Observable influence may also be called physical. Non-observable behaviour does not influence the world and may be assumed to not exist in a program.
- **Bit** is a unit of data storage capable of holding one of two values, commonly referred to as $\{0, 1\}$. Individual bits do not need to be addressable.
- A **byte** is the smallest unit of bits that is addressable.

  > Note A byte commonly consists of 8 bits though a different length can be used for a conforming implementation.

- **Binary data** is the representation of data using bits, consisting of one or more bytes.
- A **character** is a member of a set of characters that make up the textual representation of a Sonnet program.
- An **implementation-defined** value or behaviour is one left unspecified by the specification where each implementation must document its choice. A subset of this is **locale-defined** where rather than the implementation the circumstances of its invocation define certain behaviours.

  > Example The behaviour of functions such as `is_lower` may depend on the language a user has set for their computer.

- **Unspecified behaviour** is behaviour for which the specification imposes no requirements, allowing multiple valid outcomes. After the unspecified behaviour, program

execution continues normally. Unspecified behaviour is defined by explicitly declaring it as such or by the omission of any explicit definition of behaviour.

> Example The evaluation order of + is unspecified; thus
>
> ```
> print("Hello ") + print("world!")
> ```
>
> may produce either “Hello World!” or “world!Hello “.

– **Undefined behaviour** is a sub-case of unspecified behaviour for which this specification imposes no requirements. Reaching undefined behaviour completely invalidates the semantics of the entire program execution; no statements are made regarding any behaviour, including behaviour prior to the point at which the undefined behaviour was invoked. Undefined behaviour may be the result of evaluating erroneous program constructs or data.

4. Conventions

– The term *value set* is used to refer to the set of representable values for some type.

5. Syntax Specification

5.1. Textual representation

The representation form of Sonnet programs is text; a program is a sequence of characters. The set of characters must at least suffice the following requirements, being able to encode all listed characters:

– all the letters from the Latin alphabet, those being:

A B C D E F G H I J K L M
N O P Q R S T U V W X Y Z

– and their lowercase equivalents:

a b c d e f g h i j k l m
n o p q r s t u v w x y z

– all arabic digits:

0 1 2 3 4 5 6 7 8 9

– as well as the following set of special characters (“␣” denotes the invisible space character):

\+ - * / < > = ! ? & | % $
. : ; , ( ) [ ] { } _ ␣

The encoding of a program except for all string literals it might contain, is implementation-defined.

> Note At the time of writing (2026) it seems best to encode the whole file in UTF-8 unless special domain constraints would apply.

Implementations may allow to substitute Unicode signs such as “→” for their normal counterparts (e. g. “->”).

String literals shall be represented as text encoded in the UTF-8 format.

Additonally a character or sequence of character shall exist, to encode a line break; in the following chapters this will be represented by the sequence “\n”.

5.2. Grammar Rules

6. Semantics

6.1. Numeric Types

The primitive types of numbers are `i8`, `i16`, `i32`, `i64`, `u8`, `u16`, `u32`, `u64`, `f16`, `f32` and `f64`. The number types of the form in, un or fn where $n \in \mathbb{N}$ use n bits for the representation of a value of this type. n is a multiply of bit-length of a byte.

More primitive number types may be defined by an implementation.

Signed integer types. The four signed integer types are `i8`, `i16`, `i32` and `i64`.

A signed integer type in represents a value encoded using a two's-complement binary representation consisting of n bits. The set of representable values $V_{\text{i}n}$ for a type in is:

$$V_{\text{i}n} = \{x \in \mathbb{Z} \mid -2^{n-1} \leq x \leq 2^{n-1} - 1\}$$

Unsigned integer types. The four unsigned integer types are `u8`, `u16`, `u32` and `u64`.

An unsigned integer type un represents a value encoded as a standard binary number. The set of representable values $V_{\text{u}n}$ for a type un is:

$$V_{\text{u}n} = \{x \in \mathbb{Z} \mid 0 \leq x \leq 2^n - 1\}$$

Floating-point types. The three floating-point types are f16, f32 and f64. A floating-point type fn represents a value encoded using the IEEE 754 standard for floating-point arithmetic.

Correspondence between floating-point types and the formats defined by IEEE 754:

`f16` half-precision floating-point
`f32` single-precision floating-point
`f64` double-precision floating-point

6.2. Conversion between number types

Conversion between integer types. The conversion of an integer type to a larger integer type, where the target value set is a superset of the source value set, preserves the original value.

Integer to integer of equal size. The set of nonnegative values inhabiting a signed integer type is a subset of the set for the corresponding unsigned integer type, and the representation of the same value shall be identical in both types.

A negative integer a is converted to a positive unsigned integer by adding 2^n where n is the number of bits used to represent a. Conversely, converting an unsigned integer to a signed integer is done by subtracting 2^n.

> NOTE A signed value -2^{n-1} becomes 2^{n-1}, the middle of the range of integers of the correspondent unsigned type. The biggest value representable by an unsigned type, $(2^n - 1)$, become -1.

Unsigned integer to larger integer. The value remains preserved exactly in accordance to Section 6.2.1.

Signed integer to larger integer. A signed integer value a is converted to a larger target integer type of n bits by preserving its value exactly if the target type is signed or if $a \geq 0$. If $a < 0$ and the target type is unsigned, the value is converted by adding 2^n.

Integer to integer of lesser size. An integer value a is converted to a smaller target integer type of n bits by first reducing its value modulo 2^n to an integer in the range $[0, 2^n - 1]$. If the target type is signed and the resulting value is greater than or equal to 2^{n-1}, the final value is obtained by subtracting 2^n from that result.

> NOTE Operationally, this corresponds to truncating the higher-order bits, retaining only the lowest n bits of the original binary representation.
>
> If the target type is unsigned, these remaining bits are interpreted as a standard binary number. If the target type is signed, the highest bit of the remaining n bits acts as the sign bit.

Floating-point number to integer. If the floating-point numbers' integral component can be represented in the target type that shall be the result, otherwise the result is implementation-defined.

Integer to floating-point. The result of converting a value of an integer type (in or un) to a floating-point type fm is determined by whether the value can be exactly represented in the target format:

- If the integer value can be represented exactly in the target floating-point type, the result is that exact representation.
- If the integer value cannot be represented exactly but falls between two representable floating-point values, the value is rounded to one of the adjacent representable values. The choice of rounding direction is implementation-defined but must conform to the IEEE 754 standard.

– If the integer value exceeds the maximum representable finite value of the target floating-point type, the result is undefined.

Integer Overflow. If an integer overflows the behaviour is left undefined.

References